Sistemas Distribuidos

Invocación Remota

RPC

PIENSA
cree
sueña
Atrevéte
WWW.HAPPINESSPLAY.ES

Invocación Remota

Agenda

RPC

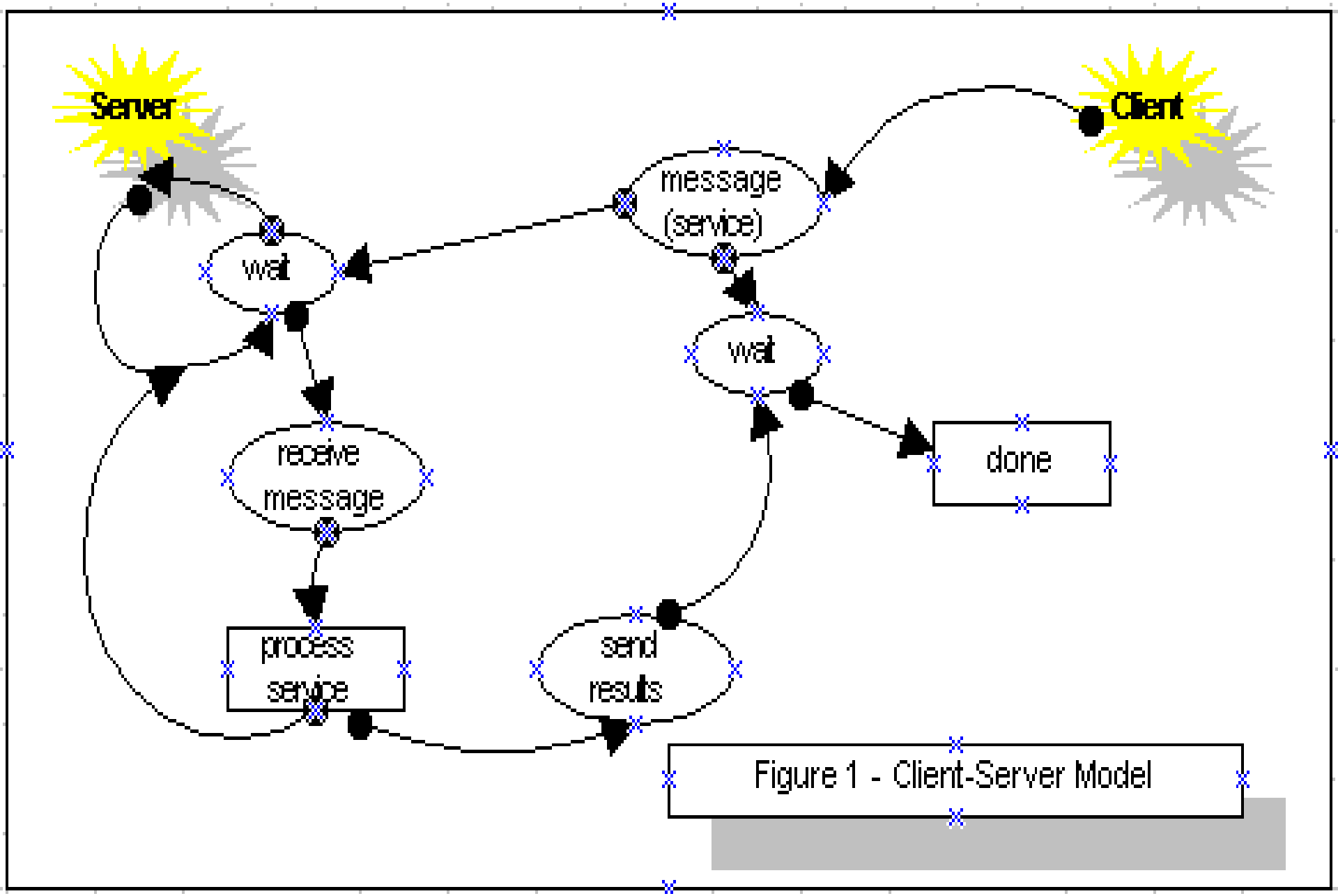
Java RMI

CORBA

RPC: Introducción

- ✓ En el modelo Cliente-Servidor la comunicación se construye en base a operaciones de E/S. Todo debe ser construido por el usuario
- ✓ La llamada a procedimiento remoto se ideó para facilitar la comunicación entre máquinas .
- ✓ Un procedimiento en la máquina *A* llama a un procedimiento en la máquina *B*. *A* se bloquea hasta que el procedimiento de *B* termine.
- ✓ El programador no se preocupa de los mensajes enviados entre *A* y *B*; la llamada a procedimiento remoto debe ser lo más parecida posible a una llamada local

Modelo Cliente-Servidor



RPC

- Se construye como una llamada normal a una subrutina.
- Puede ser de la forma

return_code = procedure name (input_parameters, output_parameters)

Al terminar el código de retorno indica si el procedimiento terminó con éxito.

Se ha construido para ser transparente o sea similar a una llamada local

Componentes RPC

- Uno o varios programas que se ejecutan en un servidor
- Cada programa puede ejecutar varios procedimientos.
- Cada procedimiento realiza una función bien definida (por ej. leer un archivo)
- Cada programa tiene un identificador numérico asignado
- Cada procedimiento tiene un identificador numérico asignado

RPC

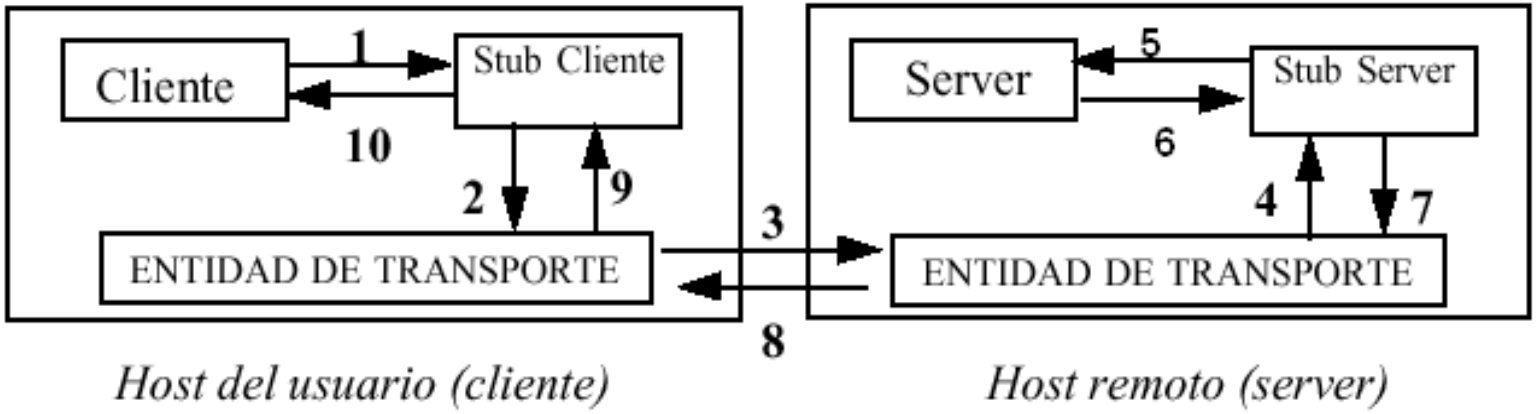
Dificultades para implementar RPC:

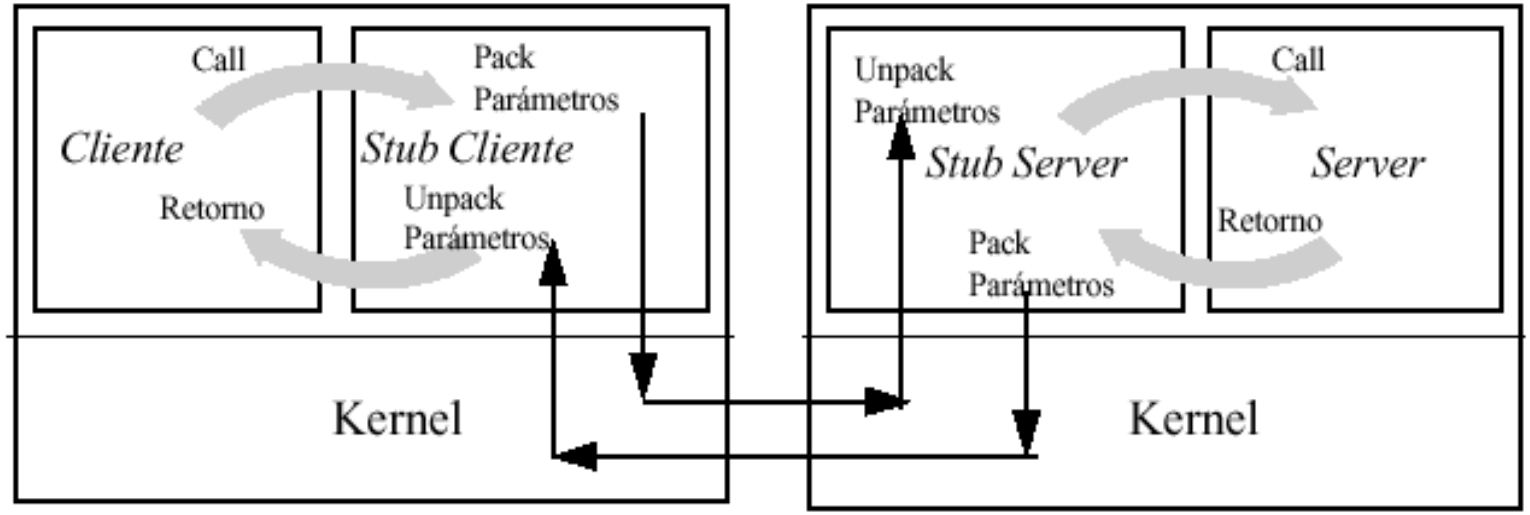
- Las máquinas utilizan espacios de direcciones distintas, ¿punteros?, ¿variables globales?
- La transferencia de parámetros y resultados, ¿son los tipos iguales en ambas máquinas?
- Fallo de alguna de las máquinas

La transferencia de parámetros se realiza codificando/decodificando a un formato de tipos prefijado

La transferencia de punteros puede hacerse por copia de zonas de memoria, pero en ciertos casos es más difícil

- **REMOTE PROCEDURE CALL**





Operación Básica de RPC

Pasos de RPC

1- Procedimiento Cliente llama cliente-stub

2- Cliente stub construye un mensaje (empaqueta) y produce un system call

3- El kernel del cliente envía mensaje kernel remoto (del server)

4- El kernel remoto entrega mensaje al server-stub

5- El server-stub desempaqueta el mensaje y llama al server

6- Procedimiento Server hace el trabajo y retorna resultados

7- Server-stub construye un mensaje (empaqueta) y produce un system call

8- El kernel del server envía un mensaje al kernel del cliente

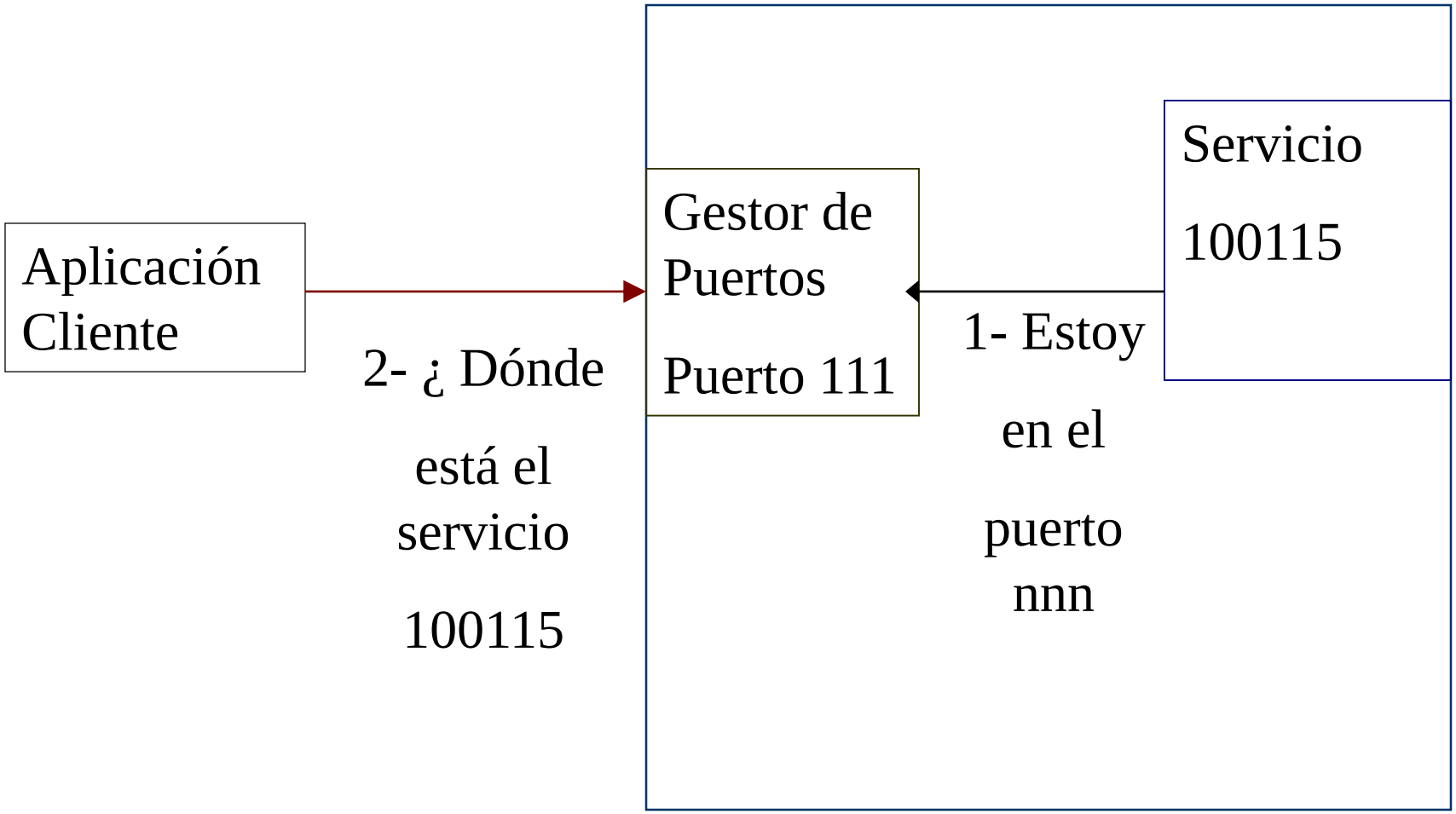
9- El kernel del Cliente entrega el mensaje al cliente-stub

10- El cliente-stub desempaqueta y retorna al cliente.

Gestor de puertos RPC

- Muchos programas servers RPC usan port efímeros, y a veces no los conocemos.
- En cada host existe un programa llamado gestor de puertos (portmapper, y en nuevas versiones rpcbind)
- Mantiene una lista de: los programas locales de RPC activos, el número de versión de los programas, el protocolo de transporte y puertos en que se ejecutan los programas

Gestor de puertos RCP



Gestor de puertos RCP

¿Qué puede suceder si RPC genera port efímeros aleatoriamente o digamos incrementando los números de ports, por ejemplo desde 1024 en adelante?

¿Qué efectos puede tener Incrementar los números de puertos en el rango de puertos efímeros de RPC?

Compiladores RPC

- Para generar aplicaciones RPC utilizamos algún compilador como Sun RPC (rpcgen) o RPC package de OSF.
- Un compilador toma una definición de procedimiento remoto escrito en lenguaje RPC y genera: archivos de encabezamiento para server y cliente, stub del cliente y del server y rutinas XDR de codificación y decodificación.

Sun RPC

- Es provisto como parte de Sun y otros OS Unix y también disponible en instalaciones NFS.
- Permiten usar UDP o TCP.
- Los mensajes están restringidos en longitud (teórico 64 Kbytes, en la práctica 8 Kbytes).
- Los mensajes proveen autenticación permitiendo elegir entre estilo Unix, propio de RPC, clave compartida o Kerberos

Consideraciones RPC

- En C en general los enteros y escalares siempre se pasa por valor, mientras que los arreglos siempre se pasan por referencia.
- Pero si queremos pasar por referencia el espacio de direcciones de cliente y servidor son distintos.
- Se utiliza una técnica llamada copia restauración donde se copian los datos del cliente al servidor en la llamada al procedimiento y si los datos se deben retornar, se copian en el cliente desde el servidor.

Consideraciones RPC

- El modelo sincrónico tradicional de RPC exige que el cliente quede bloqueado hasta que el servidor devuelva la respuesta.
- Cuando no hay resultado que devolver no es eficiente que el cliente quede bloqueado.
- Ejemplo que no necesitan bloqueo pueden ser transferencia de dinero de una cuenta a otra, agregar entrada a una base de datos, iniciar servicios remotos, el procesamiento batch, etc.

Sincrónico y Asincrónico

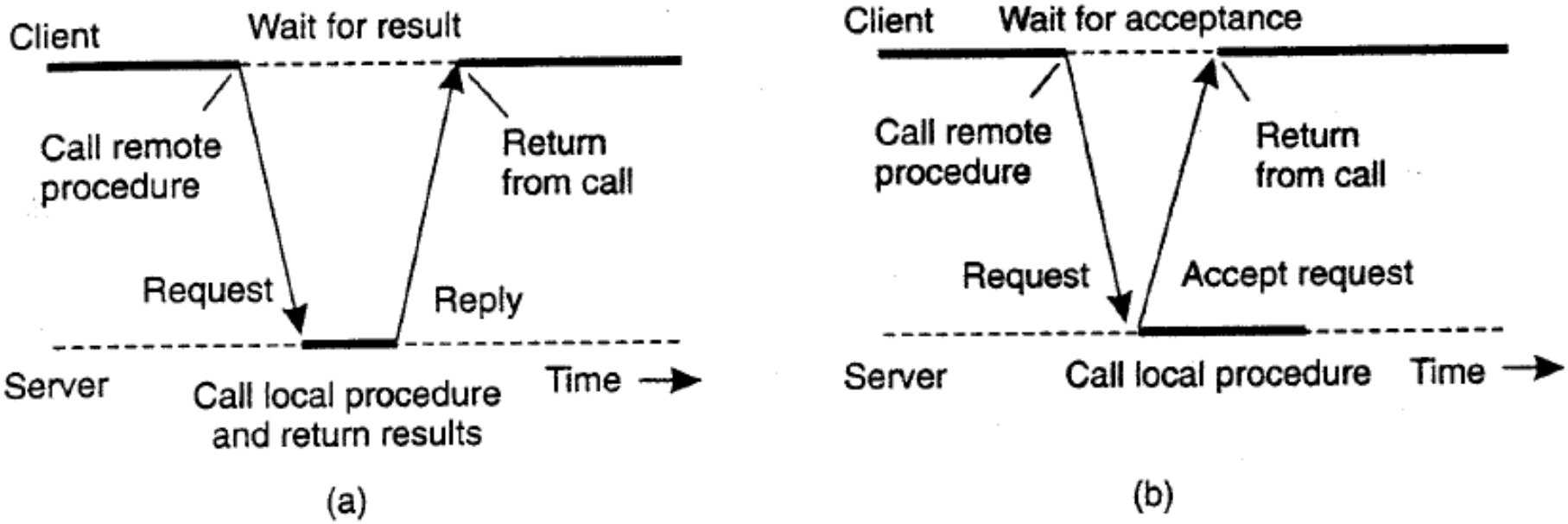


Figure 4-10. (a) The interaction between client and server in a traditional RPc. (b) The interaction using asynchronous RPc.

Consideraciones RPC

- La transparencia del lado del cliente embellece todo el esquema accediendo a los servicios remotos utilizando llamada a procedimientos ordinarios y no llamadas send y receive. Todos los detalles de mensajes se ocultan en los procedimientos de bibliotecas

Tratamiento de fallos RPC

CAÍDA DEL SERVER

Pensar ¿Qué puede suceder?

– La operación entre cliente y server se puede repetir o no dependiendo del tipo de operación:

- Idempotente (puede repetirse n veces)
- No posee la semántica Exactamente una vez
- **Posee mínimo una vez**

CAÍDA DEL CLIENTE:

El server debe resolver el problema de tratamiento de huérfanos. ¿Qué haría en este caso?

XDR

- Logrando que los datos sean portables, el programador no debe preocuparse por la representación de datos y de cometer errores de inconsistencia.
- XDR es una representación estándar de datos, provee una especificación exacta para números, strings, arreglos, estructuras y uniones. Incluye:

– un lenguaje de definición de estructuras de datos inspiradas en “C” (como ASN.1)

– una serie de reglas para codificar estructuras XDR para su transmisión (mas simples y compactas).

XDR

- Transferencia de datos:

– palabras de 4 octetos en orden big endian

- Tipo de datos:

- Constantes, enteros, reales, opacos, cadenas, arreglos, estructuras, uniones y definiciones de tipo.

- A partir de rutinas primitivas pueden construirse tipo de datos complejos
- Los datos opacos no son interpretados (datos encriptados)

Servicios RPC

Los sig. Servicios fueron escritos usando RPC y XDR:

- NFS
- Mount
- Network Lock Manager
- Network Status Monitor
- Network Information System (NIS)
- Portmapper
- Rusers (servicio de difusión)

Otros servicios RPC

- Lock Manager: es responsable de prevenir que múltiples procesos modifiquen al mismo archivo al mismo momento
- Status Monitor: es el responsable de reunir la información de estado de los otros nodos
- NIS: su función más importante es el servicio de administración de red que crea y mantiene una base de datos que contiene información de password y direcciones de toda la red

Sun RPC

- La implementaciòn Sun RPC modela las acciones como idempotentes con la semántica mínimo una vez, por lo tanto requiere adaptaciones para suprimir duplicados y proveer semántica exactamente una vez
- El procedimiento remoto solo puede aceptar un parámetro, se puede salvar utilizando una estructura como parámetro.

RPC - Conclusiones

RPC is one of several models for implementing distributed communication

It is particular useful for transparently supporting request/response-style applications

However, it is not appropriate for all applications due to its performance overhead and lack of flexibility

Before deciding on a particular communication model it is crucial to carefully analyze the distributed requirements of the applications involved

Nuevos plataformas para desarrollos RPC

- **RPyC** (pronounced *are-pie-see*), or **R**emote **Py**thon **C**all, is a Python library for RPC, as well as distributed computing. RPyC is transparent, symmetric, and requires no definition languages. (symmetric, exceptions, synchronous and asynchronous).
- **XML-RPC** is a RPC protocol which uses XML to encode its calls and HTTP as a transport mechanism
- **JSON-RPC** is a RPC protocol encoded in JSON. It is a very simple protocol (and very similar to XML-RPC), defining only a few data types and commands. JSON-RPC allows for notifications (data sent to the server that does not require a response) and for multiple calls to be sent to the server which may be answered out of order.

Nuevos plataformas para desarrollos RPC

- Thrif (Facebook - Apache): Is software framework, for scalable cross-language services development, combines a software stack with a code generation engine to build services that work efficiently and seamlessly between C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, Cocoa, JavaScript, Node.js, Smalltalk, Ocaml, Delphi and other language.
- The Internet Communications Engine - Ice (ZeroC), is an open source RPC framework. It provides SDKs for C++, C#, Java, JavaScript, Objective-C, PHP, Python, and Ruby, and can run on various OS, including Linux, Windows, OS X, and Android. Ice Touch is a variant of Ice for iOS devices, and Ice-E is another variant for embedded devices

Fin de RPC